

Fiche n°1 Structure de base d'un document HTML5(Rappel)

Les documents HTML sont des documents en texte brut enregistrés avec une extension de fichier .html ou .htm plutôt qu'une extension de fichier .txt.

Tous les documents HTML ont une structure minimale obligatoire qui comprend la déclaration et des éléments suivant: `<!DOCTYPE html>`, `<html>`, `<head>` et `<body>`.

```
<!DOCTYPE html>

<html Lang="fr">

  <head>

    <meta charset="utf-8"/> <title> Titre de la page </title>

  </head>

  <body> Contenu visible </body>

</html>
```

La déclaration du type de document, ou `<!DOCTYPE html>`, informe les navigateurs web de la version de HTML utilisé et est placée au début du document HTML. Une déclaration pour un document écrit en HTML5 est simplement `<!DOCTYPE html>`

Après la déclaration de type de document, l'élément `<html>` marque le début du document.

A l'intérieur de `<html>`, la balise `<head>` contient l'entête du document, y compris toutes les métadonnées (informations sur la page Web). Le contenu à l'intérieur de l'élément `<head>` n'est pas affiché sur la page Web elle-même. A la place, il peut contenir entre autres :

`<title>` qui est le titre de la page Web qui apparaît dans la barre des titres du navigateur. C'est le seul élément dans `<head>` qui a un effet visuel



`<link>` Qui fait référence à une ressource données comme un fichier de style CSS

`<meta>` donne des informations supplémentaires sur la page comme, l'encodage, le résumé de la page, des instructions pour les moteurs d'indexation comme Google,

`<style>` délimite du code CSS directement dans la page Web

`<script>` permet d'inclure du code JavaScript dans la page ou bien de faire référence à un fichier contenant du code JavaScript.

Le code suivant donne un exemple de <head> sachant qu'aucun de ces éléments n'est vraiment obligatoire.

```
<head>
  <meta charset="UTF-8">
  <meta name="description" content="Université Virtuelle Africaine">
  <meta name="keywords" content="UVA, AVU, Université Virtuelle,
Afrique">
  <meta name="author" content="UVA">
  <link rel="stylesheet" type="text/css" href="theme.css">
  <style>
    h1 {color:red;}
    p {color:blue;}
  </style>
  <script>
    document.getElementById("demo").innerHTML = "Hello JavaScript!";
  </script>
</head>
```

La dernière série de balises <body> et </ body>, entoure tout le contenu dans la page WEB qui est censé être affiché comme les titres, les paragraphes, les images, les tableaux etc. Il a quelques attributs intéressent comme background permet de mettre une image ou une couleur en fond de la page Web.

Fiche n°2

Syntaxe de base de HTML5(Rappel)

- ❖ HTML est un langage de balisage permettant de structurer comment le contenu de la page Web va être interprété et présenté par le navigateur.
 - ❖ Toutes les balises ont le même format : elles commencent par un signe inférieur à « < » et finissent par un signe supérieur à « > ».
- Exemple de balises : <a>,
, <html>.

Les attributs des balises

- ❖ Les balises peuvent également contenir des attributs. Les attributs des balises sont des mots qui décrivent des propriétés de l'élément, et donnent des instructions au navigateur. Les attributs d'un élément HTML sont insérés dans la balise ouvrante sous la forme générale suivante :

<balise nom_attribut="valeur" ...>

- ⇒ Le nom de l'attribut est suivi d'un signe égal = et la valeur est mise entre guillemets. La ligne suivante donne quelques exemples de balises avec attributs.
- ❖ Il existe quelques attributs qui sont communs à presque toutes les balises dont les plus utilisés sont :
 - **id** : cet attribut qui permet de donner un identifiant unique à un élément sur la page Web. Sa valeur est un texte alphanumérique fourni par le programmeur. Cet attribut est presque systématiquement utilisé dans les fichiers CSS et dans le code JavaScript.
 - **style** et **class**: ces attributs permettent d'appliquer un style CSS sur un élément. Leurs contenus seront vu dans la partie CSS de ce chapitre
 - **title** : Cet attribut permet d'ajouter une information supplémentaire sur un élément. Sa valeur est souvent présentée comme un texte infobulle par le navigateur.
- ❖ Le code HTML a quelques règles qui sont importantes à connaître :
 - Insensible à la casse : HTML ne fait pas de différence entre les majuscules et les minuscules dans le nom des balises. est donc équivalent à . Toutefois dans le contenu HTML un texte en majuscule (resp. minuscule) reste en majuscule (resp. minuscule) à moins que vous ayez explicitement spécifié une transformation de casse dans le code. L'insensibilité à la casse opère dans l'interprétation des balises et de leurs attributs.
 - Balises imbriquées : Lorsque des balises sont imbriquées les uns dans les autres, ils doivent suivre une règle simple : Si une balise est ouverte, toutes les balises en pair qui sont dans son contenu doivent être fermées avant que sa balise fermante.
- ❖ Quelques balises auto fermantes:
, <embed>, <hr>, , <input>, <link>, <meta>, <param>, <source>, <wbr>

❖ **Balises de premier niveau**

Les balises de premier niveau sont les principales balises qui structurent une page HTML. Elles sont indispensables pour réaliser le « code minimal » d'une page web.

- **<html>** Balise principale
- **<head>** En-tête de la page
- **<body>** Corps de la page

❖ **Balises d'en-tête**

Ces balises sont toutes situées dans l'en-tête de la page web, c'est-à-dire entre `<head>` et `</head>` :

- **<link />** Liaison avec une feuille de style
- **<meta />** Métadonnées de la page web (charset, mots-clés, etc.)
- **<script>** Code JavaScript
- **<style>** Code CSS
- **<title>** Titre de la page

❖ **Balises de structuration du texte**

• <abbr> Abréviation	• <blockquote> Citation (longue)
• <cite> Citation du titre d'une œuvre ou d'un évènement	• <q> Citation (courte)
• <sup> Exposant	• <sub> Indice
• Mise en valeur forte	• Mise en valeur normale
• <mark> Mise en valeur visuelle	• <h1> Titre de niveau 1
• <h2> Titre de niveau 2	• <h3> Titre de niveau 3
• <h4> Titre de niveau 4	• <h5> Titre de niveau 5
• <h6> Titre de niveau 6	• Image
• <figure> Figure (image, code, etc.)	• <figcaption> Description de la figure
• <audio> Son	• <video> Vidéo
• <source> Format source pour les balises <audio> et <video>	• <a> Lien hypertexte
•
 Retour à la ligne	• <p> Paragraphe
• <hr /> Ligne de séparation horizontale	• <address> Adresse de contact
• Texte supprimé	• <ins> Texte inséré
• <dfn> Définition	• <kbd> Saisie clavier
• <pre> Affichage formaté (pour les codes sources)	• <progress> Barre de progression
• <time> Date ou heure	

❖ **Balises de listes**

Cette section énumère toutes les balises HTML permettant de créer des listes (listes à puces, listes numérotées, listes de définitions...)

	Liste à puces, non numérotée		Liste numérotée
	Élément de la liste à puces	<dl>	Liste de définitions
<dt>	Terme à définir	<dd>	Définition du terme

❖ Balises de tableau

<table>	Tableau	<caption>	Titre du tableau
<tr>	Ligne de tableau	<th>	Cellule d'en-tête
<td>	Cellule	<thead>	Section de l'en-tête du tableau
<tbody>	Section du corps du tableau	<tfoot>	Section du pied du tableau

❖ Balises de formulaire

<form>	Formulaire	<fieldset>	Groupe de champs
<legend>	Titre d'un groupe de champs	<label>	Libellé d'un champ
<input />	Champ de formulaire (texte, mot de passe, case à cocher, bouton, etc.)	<textarea>	Zone de saisie multiligne
<select>	Liste déroulante	<option>	Élément d'une liste déroulante
<optgroup>	Groupe d'éléments d'une liste déroulante		

❖ Balises sectionnantes

Ces balises permettent de construire le squelette de notre site web.

- | | |
|---|---|
| <ul style="list-style-type: none">• <header> En-tête• <nav> Liens principaux de navigation• <footer> Pied de page• <section> Section de page | <ul style="list-style-type: none">• <article> Article (contenu autonome)• <aside> Informations complémentaires |
|---|---|

❖ Balises génériques

Les balises génériques sont des balises qui n'ont pas de sens sémantique.

En effet, toutes les autres balises HTML ont un sens : **<p>** signifie « Paragraphe », **<h2>** signifie « Sous-titre » etc.

Parfois, on a besoin d'utiliser des balises génériques (aussi appelées balises universelles) car aucune des autres balises ne convient. On utilise le plus souvent des balises génériques pour construire son design.

Il y a deux balises génériques : l'une est inline, l'autre est block.

- **** Balise générique de type inline
- **<div>** Balise générique de type block

Ces balises ont un intérêt uniquement si vous leur associez un attribut class, id ou style :

- **class**: indique le nom de la classe CSS à utiliser.
- **id**: donne un nom à la balise. Ce nom doit être unique sur toute la page car il permet d'identifier la balise. Vous pouvez vous servir de l'ID pour de nombreuses choses, par exemple pour créer un lien vers une ancre, pour un style CSS de type ID, pour des manipulations en JavaScript, etc.
- **style**: cet attribut vous permet d'indiquer directement le code CSS à appliquer. Vous n'êtes donc pas obligés d'avoir une feuille de style à part, vous pouvez mettre directement les attributs CSS. Notez qu'il est préférable de ne pas utiliser cet attribut et de passer à la place par une feuille de style externe, car cela rend votre site plus facile à mettre à jour par la suite.

Ces trois attributs ne sont pas réservés aux balises génériques : vous pouvez aussi les utiliser sans aucun problème dans la plupart des autres balises.

LES FORMULAIRES

Les pages Web construites en HTML permettent de présenter et de diffuser de l'information en provenance d'un serveur Web vers un navigateur client. Un « dialogue » client-serveur s'instaure lorsque le client peut à son tour envoyer des informations au serveur, notamment par le biais de formulaires :

1. recueil d'informations à l'aide de contrôles dans un formulaire HTML,
2. envoi des informations au serveur,
3. traitement des informations par le serveur (à l'aide d'un langage adapté tels que PHP ou Perl),
4. renvoi éventuel d'informations HTML au client (réponse, accusé de réception, demande de précision...).

Une page peut comprendre un à plusieurs formulaires dans sa section `<body>`. Tous les contrôles du formulaire doivent être insérés entre les balises **`<form>...</form>`** :

Balise	Description	Attributs facultatifs
<form>	formulaire contenant des contrôles	name="nomFormulaire" method="get post" méthode d'envoi des données au serveur (get par défaut). action ("URL") adresse du script auquel les données sont envoyées (pour être traitées). Un à plusieurs évènements JavaScript peuvent être spécifiés

Une zone de texte permet de saisir tout type de données. La voici en version « minimale » :

contrôle de formulaire	code HTML
La zone de texte : Nom :	Nom : <code><input type="text" name="nom" /></code>

L'attribut **name** est obligatoire afin de permettre au serveur, chargé de traiter les valeurs, de récupérer ces valeurs à partir du nom qui leur est attribué (**deux contrôles d'un même formulaire ne peuvent donc pas avoir la même valeur de l'attribut name**).

La version « maximale » :

Diagram illustrating the attributes of the `<input type="text" name="nom" id="idNom" size="20" maxlength="30" placeholder="indiquez votre nom" autofocus required />` tag:

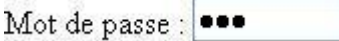
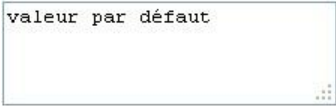
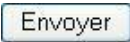
- `autofocus`: zone sélectionnée par défaut (une par page)
- `required`: contenu requis (validation du formulaire empêchée si la zone est vide)
- `size="20"`: largeur affichée (en nombre de caractères) de la zone
- `maxlength="30"`: nombre de caractères maximal saisissable
- `placeholder="indiquez votre nom"`: message d'aide à la saisie

Le message issu du **placeholder** disparaît lorsque le curseur est placé dans la zone (ce qui est d’emblée le cas ici du fait de l’autofocus) : il apparaît lorsque la zone n’est pas sélectionnée et qu’aucun contenu n’a été saisi. **placeholder**, **autofocus** et **required** sont des attributs HTML5 pas encore reconnus par tous les navigateurs.

L'attribut **default** permet d'indiquer une valeur par défaut dans la zone (que l'arrivée du curseur dans la zone ne supprime pas) : s'il n'y a pas de ressaisie, c'est cette valeur qui sera transmise (alors que placeholder n'indique pas une valeur).

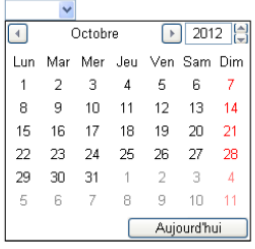
La balise **<label>** a une portée sémantique (repérer les libellés de formulaire), son attribut **for** se réfère à l'id de la zone de texte (qui peut porter la même valeur que **name**) permet d'envoyer le curseur dans la zone quand on clique sur le libellé.

Présentons les contrôles de formulaire :

contrôle de formulaire	code HTML
La zone de champ caché : elle peut servir à stocker (et transmettre) des valeurs qui n'ont pas besoin d'être affichées à l'écran (mais visibles dans le code source).	<code><input type="hidden" name="c" value="caché" /></code>
La zone mot de passe : 	Mot de passe : <code><input type="password" name="mp" /></code>
La zone de texte long : 	<code><textarea type="text" name="lib" rows="3" cols="25">valeur par défaut</textarea></code> L'apparence de la zone de texte multilignes est paramétrable soit à l'aide des attributs <code>rows</code> (lignes) et <code>cols</code> (caractères en elle permet de saisir tout type de données, sur plusieurs lignes colonnes) ou en CSS (propriétés <code>width</code> et <code>height</code>).
Les boutons radio : Sexe : ☒ Homme ☐ Femme ils permettent d'effectuer <u>un seul choix</u> (conditionné par une valeur identique pour l'attribut <code>name</code>) parmi des propositions L'attribut facultatif <code>checked</code> détermine la valeur préselectionnée.	Sexe : <code><input type="radio" name="sexe" value="M" checked /> Homme</code> <code><input type="radio" name="sexe" value="F" /> Femme</code>
Les cases à cocher : Jour(s) de disponibilité : ☐ Mercredi ☐ Samedi ☐ Dimanche elles permettent d'effectuer un à plusieurs choix parmi un nombre limité de propositions ; les choix effectués peuvent être stockés dans un tableau (attribut <code>name</code> ci-contre) ou chaque case peut avoir sa propre valeur de l'attribut <code>name</code>	Jour(s) de disponibilité : <code><input type="checkbox" name="jour[]" value="1" /> Mercredi</code> <code><input type="checkbox" name="jour[]" value="2" /> Samedi</code> <code><input type="checkbox" name="jour[]" value="3" /> Dimanche</code>
La liste déroulante : Statut :  elle permet d'effectuer <u>un seul choix</u> parmi un nombre élevé de propositions	Statut : <code><select name="statut"></code> <code><option value="1">Etudiant</option></code> <code><option value="2">Professeur</option></code> <code></select></code> L'attribut <code>value</code> indique la valeur renvoyée.
La zone de liste : Jour(s) de disponibilité :  elle permet d'afficher d'emblée plusieurs (selon l'attribut <code>size</code>) propositions. L'attribut facultatif <code>multiple</code> permet le choix multiple (l'attribut <code>name</code> doit alors désigner un tableau)	Jour(s) de disponibilité : <code><select size="3" multiple name="jour[]"></code> <code><option value="l">lundi</option></code> <code><option value="m">mardi</option></code> <code><option value="j">jeudi</option></code> <code><option value="v">vendredi</option></code> <code></select></code> L'attribut facultatif <code>selected</code> de la balise <code><option></code> permettrait de préselectionner un élément de la liste.
Le fichier joint : PJ :  il permet de rechercher un fichier à envoyer en pièce jointe	PJ : <code><input type="file" name="pj" value="fichier" /></code>
Le bouton Annuler :  il réinitialise tous les contrôles du formulaire à vide ou à leur valeur par défaut	<code><input type="reset" value="Annuler" /></code>
Le bouton Envoyer :  il envoie les données du formulaire au serveur pour un traitement par le script spécifié dans l'attribut <code>action</code> de la balise <code><form></code>	<code><input type="submit" value="Envoyer" /></code>

Les zones de saisie enrichies

Les zones de type `tel`, `mail` et `url` vérifient que le contenu saisi respecte le format requis et signalent l'erreur si ce n'est pas le cas (empêchant la validation du formulaire) :

type="date"	type="number"	type="range"	type="color"
 il existe également des types <code>time</code>, <code>datetime</code>, <code>week</code> et <code>month</code>	 les attributs <code>min</code>, <code>max</code> permettent de préciser un minimum et un maximum, <code>step</code> un pas d'évolution entre deux nombres <pre><input type="number" min="0" max="9" step="1.5" value="3" /></pre>		 choix parmi 16 millions de couleurs

Regroupement de champ du formulaire par thématique

Liste (déroulante ou non) avec groupements d'options extrait du code HTML	affichage par un navigateur
<pre><select size="9" name="jour"> <optgroup label="semaine"> <option value="1">lundi</option> <option value="2">mardi</option> <option value="3">mercredi</option> <option value="4">jeudi</option> <option value="5" selected>vendredi</option> </optgroup> <optgroup label="week end"> <option value="6">samedi</option> <option value="7">dimanche</option> </optgroup> </select></pre>	 Les libellés de groupes (semaine et week-end) ne sont pas cliquables

Script HTML	Affichage	Liste de suggestions Ce type de valeur est le plus souvent alimenté à partir de valeurs déjà enregistrées dans une base de données.
<pre><input type="text" list="serie"> <datalist id="serie"> <option value="juillet"> <option value="août"> </datalist></pre>	 Rq : si août est saisi il sera ensuite proposée deux fois (une fois par la liste de suggestion, une fois par la mémoire cache du navigateur).	

extrait du code HTML	affichage par un navigateur
<pre><fieldset> <legend>identité</legend> <table> <tr><td><label for="idNom">Nom :</label></td> <td><input type="text" name="nom" /></td> </tr> <tr> <td><label for="idPrenom">Prenom :</label></td> <td><input type="text" name="prenom" /></td> </tr> </table> </fieldset></pre>	

Remarque :

Output		<code><output onload="value = 2 + 2"></output></code>
Progress		<code><progress id="prog" max=100 value=75> 75%</progress></code>

L'apparence des messages de validation est personnalisables à l'aide de pseudo-classes CSS : `:required` | `:optional`, `:valid` | `:invalid` et `:in-range` | `:out-of-range`.

Exercice1 :

This is my form

Email:

Password:

Confirm Password:

Upload Your Picture:

Browse...

Gender:

☐ Male ☐ Female

Select Country:

United Kingdom

Hobby:

☐ Cricket ☐ Football

Address

Submit

Exercice2 :

Step 1: Your details

Name

First and last name

Email

example@domain.com

Phone

Eg. +447500000000

Step 2: Delivery address

Address

Post code

Country

Step 3: Card details

Card type

☐  VISA

☐  AmEx

☐  Mastercard

Card number

Security code

Name on card

Exact name as on the card

BUY IT!

3^{ème} SI

Systèmes & Technologies Informatiques

Fiche n°4

Syntaxe de base CSS 3 (Rappel)C'est quoi le CSS ?

- ❖ Le CSS est un langage qui permet de styliser une page HTML à l'aide d'un sélecteur, d'une propriété et d'une valeur.
- ❖ Contrairement à l'HTML qui se reconnaît avec ses balises d'ouverture et de fermeture, le CSS lui, se reconnaît simplement avec ses classes, une accolade précédant ses propriétés et un double-point précédant ses valeurs et une accolade fermante.

Comment appliquer le CSS a une page HTML ?

- ❖ **Inline**: directement dans la balise html avec l'attribut style.

```
<body style="background-color:black;">
<h1 style="color:white;padding:30px;"> Tutorials </h1>
```

- ❖ **Embedded (interne)**: directement dans le fichier html avec la balise <style> .

```
<head>
<style type="text/css">
  h1 {color:white; font-size: 10px;}
  .center {display: block; margin: 0 auto;}
</style>
</head>
```

- ❖ **Linked (externe)**: dans un fichier .css externe et lié via la balise link.

```
<head>
  <link rel="stylesheet" type="text/css" href="style.css" />
</head>
```

C'est quoi un sélecteur ?

- ❖ Un sélecteur est un "nom" qui indique à quels éléments d'une page HTML on desire appliquer une règle.
- ❖ Chaque règle CSS commence nécessairement par un sélecteur

Syntaxe pour une règle CSS ?

selecteur(s) { propriété: valeur ; propriété: valeur1, valeur2, valeur3 ;}

Il existe plusieurs méthodes pour identifier les éléments, par exemple:

- ❖ on indique le nom d'une balise.
- ❖ on indique la classe d'une balise.
- ❖ on indique l'identificateur d'une balise.
- ❖ on indique le nom d'une balise plus sa classe, etc.

Fiche n°5 Les attributs HTML class , id , les sélecteurs CSS,les pseudo-classe

Les attributs HTML id et class ?

- ❖ Ces deux attributs sont particuliers en HTML puisqu'ils n'ont pas été créés pour préciser le fonctionnement d'un élément HTML en particulier (ce qui est normalement le rôle de tout attribut HTML) mais vont être principalement utilisés pour cibler certains éléments HTML et leur appliquer des styles en CSS.
- ❖ Les attributs HTML class et id sont des attributs dits globaux car on peut les ajouter dans la balise ouvrante de n'importe quel élément HTML.
- ❖ Ces deux attributs vont être principalement utilisés dans un but de mise en forme : ils nous servir à appliquer des styles CSS aux éléments qui vont les contenir.

Comment Utiliser l'attribut « id » en HTML et en CSS ?

⇒ L'attribut universel **id** définit un identifiant qui doit être unique pour l'ensemble du document. Le but de cet attribut est de pouvoir identifier un élément lorsqu'on crée un lien, avec un fragment et qu'on souhaite le manipuler avec un script ou qu'on le met en forme avec CSS.

Exemple

❖ Code HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link href="style.css" rel="stylesheet" />
  </head>
  <body>
    <h1 id="orange">Exemple Utilisation des id</h1>
    <p id="bleu"> Un premier paragraphe </p>
    <p id="rouge"> Un autre paragraphe avec un <a href="#">lien</a></p>
    <p id="gros">Un troisième paragraphe</p>
  </body>
</html>
```

❖ Code CSS (fichier style.css)

```
#orange{
  color: orange;
}
#bleu{
  color: blue;
}
#rouge{
  color: red;
}
#gros{
  font-size: 24px;
}
```

Exemple Utilisation des id

Un premier paragraphe

Un autre paragraphe avec un [lien](#)

Un troisième paragraphe

Résultat Page HTML

⇒ Pour cibler un **id** en particulier en CSS, on utilisera le symbole dièse (#) suivi de la valeur de l'**id** auquel on souhaite lier des styles.

Comment Utiliser l'attribut « class » en HTML et en CSS ?

- ⇒ L'un des grands intérêts de l'attribut HTML **class** est qu'un même attribut (et donc les styles CSS liés) va pouvoir être partagé par différents éléments. Cela facilite grandement la gestion des styles de nos fichiers HTML et nous permet de gagner beaucoup de temps.
- ⇒ En HTML5, l'attribut **class** peut servir dans n'importe quel élément HTML.
- ⇒ Pour cibler un **class** en particulier en CSS, on utilisera le symbole point (.) suivi de la valeur du class.

Valeurs d'attribut class

Valeur	Description
nomClass	• Spécifie un ou plusieurs noms de classe d'un élément. • Pour spécifier plusieurs classes, séparez les noms de classe par un espace par exemple <code></code>. Ce qui vous permet de combiner plusieurs classes CSS pour un élément HTML. Règles de nomination : • Doit commencer par une lettre de A à Z ou a-z • Peut être suivie : de lettres (A-Za-z), de chiffres (0-9), des tirets ("-") et de traits de soulignement (" _ ") • Comme tous les attributs en HTML, il est insensible à la casse

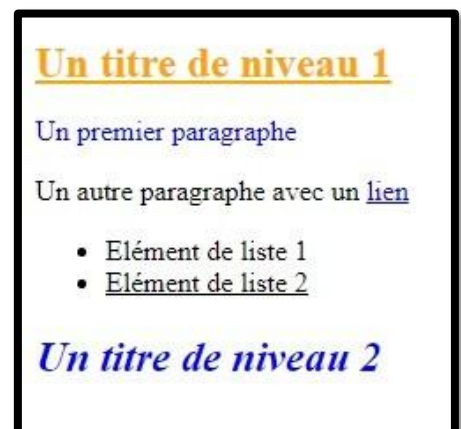
Exemple

❖ Code HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
<link href="style.css" rel="stylesheet">
</head>
<body>
  <h1 class="gros souligne orange">Un titre de niveau 1</h1>
  <p class="bleu">Un premier paragraphe</p>
  <p>Un autre paragraphe avec un <a href="#">lien</a></p>
  <ul>
    <li>Elément de liste 1</li>
    <li class="souligne">Elément de liste 2</li>
  </ul>
  <h2 class="gros italique bleu">Un titre de niveau 2</h2>
</body>
</html>
```

❖ Code CSS (fichier style.css)

```
.gros{
  font-size:24px;
}
/*Texte en italique*/
.italique{
  font-style:italic;
}
/*Crée un trait de soulignement
sous le texte*/
.souligne{
  text-decoration:underline;
}
.orange{
  color:orange;
}
.bleu{ color:blue; }
```



Résultat Page HTML

C'est quoi la pseudo-classe ?

1. Le sélecteur de lien **:hover**

C'est une *pseudo-classe* qui permet de spécifier l'apparence d'un élément au moment où l'utilisateur le survole avec le pointeur, sans nécessairement l'activer. Il s'agit en fait d'une mise en forme ciblée.

Exemple :

- **CSS :**

```
a: hover {background-color: gold; }  
p: hover {color: red; }
```

- **HTML :**

```
<a href="#"> Le fond de ce lien devient doré lorsqu'il est survolé par le  
pointeur.</a>  
<p> Ce paragraphe devient écrit en rouge.</p>
```

En outre, les liens peuvent être stylés différemment en fonction de leur **état**.
Les quatre états des liens sont :

- **a:link** - un lien normal et non visité
- **a:visited** - un lien que l'utilisateur a visité
- **a:hover** - un lien lorsque l'utilisateur passe la souris dessus
- **a:active** - un lien au moment où il est cliqué

Exemple :

- **CSS :**

```
a:link { color: red; } /* lien non visité */  
a:visited { color: green; } /* lien visité */  
a:hover { color: hotpink; } /* survol de lien */  
a:active { color: blue; } /* lien cliqué */  
  
a.classe1:hover {font-size: 20px; } /* marche aussi sans le "a" */
```

- **HTML :**

```
<a href="#">Lien1</a>  
<br />  
<a href="#" class="classe1">Lien2</a>
```

Remarque:

Pour éviter des effets de bord problématiques, on placera la règle **:hover** après les règles **:link** et **:visited** mais avant la règle **:active** (l'ordre est **:link — :visited — :hover — :active** – un moyen mnémotechnique est de se souvenir des initiales **LVHA**).

2. Mise en page des différents périphériques d'affichage : **meta name= "viewport"**

La balise `<meta name="viewport">` permet de définir les dimensions d'une page web. La balise permet de définir la largeur de la page, la hauteur et même le zoom sur cette page. Cette balise est utile pour l'affichage d'une page sur mobile (exemple : iPhone, iPod ...).

Le **Viewport** désigne schématiquement la surface de la fenêtre du navigateur.

Fiche n°6

Les unités et le Box Model en CSS3

Les unités en CSS ?

Les CSS proposent plusieurs unités possibles pour exprimer tailles et longueurs. Le W3C les répartit en unités absolues et unités relatives :

- ⇒ **Les unités absolues** : Il existe des unités en CSS qui ne sont influencées par aucune autre dimension. Appelées unités absolues, elles sont généralement utilisées pour traduire une longueur, par exemple un padding, une hauteur de bloc, la taille des caractères.

Les unités absolues en CSS	
px	Les pixels
in	Les pouces
cm	Les centimètres
mm	Les millimètres
pc	Les picas
pt	Les points

Exemple

```
.box {
  width : 150 px ;
  height : 150 px ;
}
```

L'élément **box** aura la même dimension (150 x 150 pixels), quelle que soit la taille de l'écran.

- ⇒ **Les unités relatives** : À l'opposé des unités absolues, il existe des unités qui font varier la taille d'un élément en fonction de la taille de la police ou de l'élément parent. Ces unités dites relatives au texte sont de plus en plus utilisées pour harmoniser les éléments d'un design. On trouve :

Les unités relatives en CSS	
em	Ou cadratin, unité proportionnelle à la taille de la police de l'élément parent ou du document
rem	L'unité rem fait toujours référence à la taille de la police de l'élément racine.
ex	Cette unité est relative à la hauteur de la police actuelle en minuscule.
ch	Cette unité est elle aussi peu utilisée, elle est relative à la largeur du caractère "0".
%	Relatives à la dimension du conteneur parent.

Par défaut, 1 em = 16 px si aucune taille de police n'est définie.

Exemple

```
.box {
  width : 50%;
  Height: 50%;
}
```

Le pourcentage fera référence à la largeur calculée de l'élément parent de la boîte. Ici l'élément **box** prend 50% de la taille de son élément conteneur.

- ⇒ Les CSS offrent plusieurs autres unités comme les unités angulaires (**deg**, **grad**, **rad**, **turn**), les Unités temporelles(**s**, **ms**), les Unités de résolution (**dpi**, **dpcm**, **dppx**), les unités relatives au viewport etc...

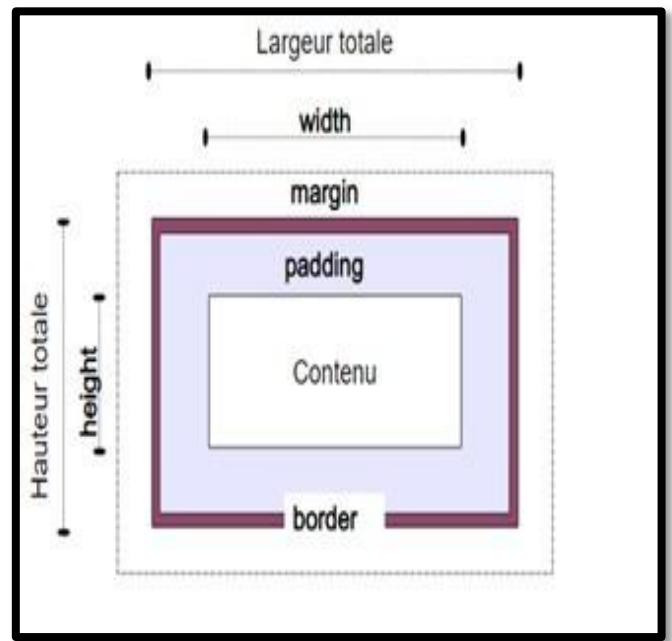
Box Model en CSS ?

⇒ Les CSS définissent un modèle de boîte (**box model**) standard pour tous les éléments HTML.

Avec:

Largeur totale= width+padding+margin

Hauteur totale=height+padding+margin



❖ Le contenu est caractérisé par sa hauteur (**height**) et sa largeur (**width**).

❖ On distingue deux types de marges :

○ Les marges intérieures « **padding** »

○ Les marges extérieures « **margin** »

❖ Le **padding** et le **margin** sont séparés par une bordure (border).

❖ Le **padding** est l'espace entre la bordure et le contenu html réel de l'élément.

❖ Le **margin** est l'espace entre la bordure de l'élément et le reste de contenu du page web.

⇒ Chaque dimension (hauteur, largeur, marges) peut être traitée de manière indépendante grâce aux mots clés : **top**, **right**, **bottom**, **left**.

Exemple:

```
.box { margin-left: 1em;
      margin-bottom: 1em;
      border-right: 1em;
}
```

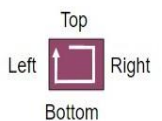
⇒ On peut utiliser la forme courte pour définir les dimensions (**margin** et **padding**) en respectant le sens de rotation horaire

Exemple

```
.box {
  margin-top: 1em;
  margin-right: 0;
  margin-bottom: 2em;
  margin-left: 0.5em;
}
```



```
.box {
  margin: 1em 0 2em 0.5em;
}
```



Types Box Model en CSS ?

⇒ Les CSS offrent plusieurs types de Box Model parmi eux on peut citer.

1) Type **inline** (boîte en ligne)

Pour une boîte de type inline les propriétés **width** and **height** ne sont pas stylables, la propriété **margin** n'aura d'effets que sur les marges gauches et droites.

2) Type **Block** (boîte en bloc)

Toutes les propriétés sont stylables (modifiables).

Exemple Box model

box.html

```
<!DOCTYPE html>
<head>
  <link href="stylebox.css" rel="stylesheet">
</head>
<body>
  <div class="main">CSS Box-Model </div>
  <!-- Class main qui contient le titre -->
  <div id="box">Exemple Box Model</div>
  <!-- id box pour le model box -->
</body>
</html>
```

stylebox.css

```
.main {
  font-size:32px;
  font-weight:bold;
  text-align:left;
}
#box {
  padding-top:40px;
  width: 400px;
  height: 100px;
  border: 50px inset green;
  margin: 50px;
  text-align:center;
  font-size:32px;
  font-weight:bold;
}
```

➡ Résultat

CSS Box-Model

Exemple Box Model

Création d'un site web interactif

Fiche n°7

MEDIAS

Les médias en HTML et CSS ?

1. <figure>

La balise **<figure>** spécifie un contenu autonome comme des illustrations, des diagrammes, des photos, des listes de codes, etc ...

L'élément **<figcaption>** ajoute une légende à l'élément **<figure>**.

<figcaption> peut être placé comme le premier ou le dernier élément de **<figure>**.

Exp. :

```
<figure>
  
  
  
  <figcaption>Photos : 3 différents logos.</figcaption>
</figure>
```

2. <video>

L'élément HTML **<video>** intègre un contenu vidéo dans un document.

Exp. :

```
<video controls="true" width="250" src="video.mp4"
poster="photo.jpg" >
</video>
```

Remarque : Si l'attribut « **controls** » n'est pas indiqué, la vidéo n'inclura pas les contrôles par défaut du navigateur.

src	L'URL du conteneur de la vidéo.
width, height	Les dimensions de l'image.
poster	L'URL d'une image à afficher en remplacement de la vidéo, en attendant qu'elle soit disponible. On utilise un frame de la vidéo.
autobuffer	Vaut <i>true</i> ou <i>false</i> , la valeur vrai déclenche le chargement en mémoire de la vidéo en même temps que la page, une option choisie quand on suppose qu'elle sera obligatoirement vue.
autoplay	Vaut <i>true</i> ou <i>false</i> , la valeur vrai démarre la vidéo quand la page est chargée.
loop	Vaut <i>true</i> ou <i>false</i> , la valeur vrai fait tourner la vidéo en boucle.
controls	Vaut <i>true</i> ou <i>false</i> , la valeur vrai indique que la barre de contrôle par défaut s'affiche, sinon le site définit sa propre barre.

• Les CODECs :

Le codec est l'algorithme de **CO**mpression et **DE**Compression de la vidéo (ou **CO**dage et **DE**Codage).

➔ Mettre à jour vos navigateurs pour avoir les codecs à jour.

Exemples de codecs : Theora, H.264, VP8 et VP9, MPEG 4, ...

Extensions utilisées : .webm, .mp4, .ogg, .avi, .mkv, ...

1. Image Mappée, <map> et <area> :

L'élément HTML **<map>** est utilisé avec des éléments **<area>** afin de définir une image cliquable divisée en régions.

Dans la balise **<map>**, on met les balises **<area>** qui définissent les zones cliquables dans une image.

Remarque : L'attribut « usemap » de la balise est associé et identique à l'attribut « name » de la balise <map>. Il crée une relation entre l'élément <map> et l'élément .

Attribut de la balise <area>

Attribut	Valeur	Description
alt	texte	Texte alternatif à la zone <area>. Obligatoire si href est présent
coords	coordonnées	Indiquez les coordonnées de la géographie
href	url	Indiquez le lien hypertexte
shape	default rect circle poly	Indiquez la forme de votre zone <area>
target	_blank _parent _self _top nom du iframe	Indiquez le mode d'ouverture (nouvelle onglet ...) du lien

Exemple 1 :

```

<map name="map1">
  <area href="page1.html" shape="rect" coords="0,0,120,120">
</map>
```

Exemple 2 :

```

<map name="map1">
  <area href="page1.htm" shape="circle" coords="240,180,140" >
  <area href="page2.htm" shape="rect" coords="50,320,450,520">
</map>
```

Exemple 3 :

```

<map name="infographic">
  <area shape="rect" coords="130,147,200,107 "
    href="https://developer.mozilla.org"target="_blank" />
  <area shape="poly" coords="130,147,200,107,130,4,59,107"
    href="https://developer.mozilla.org"target="_blank" />
  <area shape="circle" coords="145,25,60" href="page.htm" target="cible" />
</map>
```

Remarque : On peut utiliser GIMP pour créer une image mappée : filtre/web/image cliquable web

1. <iframe>

iframe signifie « inline frame », en français cadre en ligne, il s'insère dans le flot du contenu de la page.

Le tag <iframe> crée une zone intégrée (frame) sur la page Web, dans laquelle on peut afficher le contenu d'un autre document ou page web.

Attribut	Valeur	Description
height	pixels %	Spécifie la hauteur d'un iframe.
width	pixels %	Spécifie la largeur d'un iframe.
frameborder	0 1	Spécifie s'il y a une bordure autour d'un cadre ou non.
name	name	Spécifie le nom d'un cadre.
src	URL	Spécifie l'URL du document pour montrer dans un cadre.

Exemple 1 :

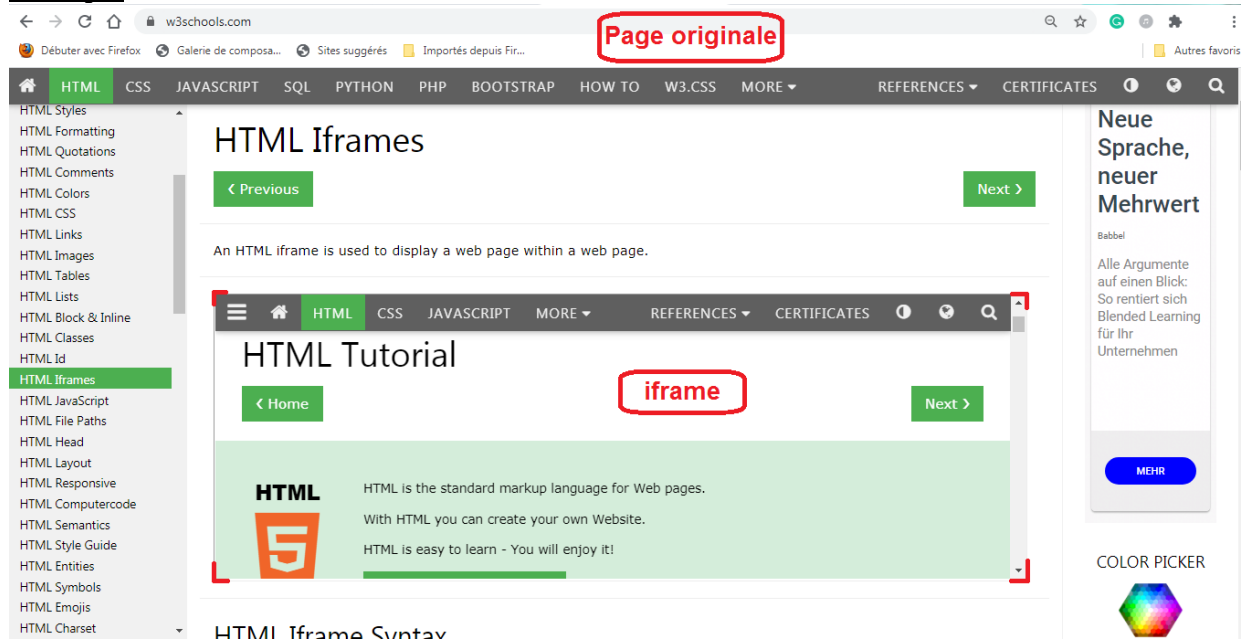
```
<iframe src="page_par_defaut.html" name="cadre1" width="320" height="200"
frameborder=0 >
Message à afficher si iframe n'est pas supporté par le navigateur
</iframe>
```

Exemple 2 :

```
<iframe src="cours.pdf" name="cadre2" width="500" height="800" frameborder=1 >  
Message à afficher si iframe n'est pas supporté par le navigateur.  
</iframe>
```

Remarque : on peut insérer dans iframe une page web, un pdf, une video, ...

Exemple:



Création d'un site web interactif

Fiche n°8

LE POSITIONNEMENT

Le positionnement CSS consiste à contrôler le placement des éléments dans une page Web.

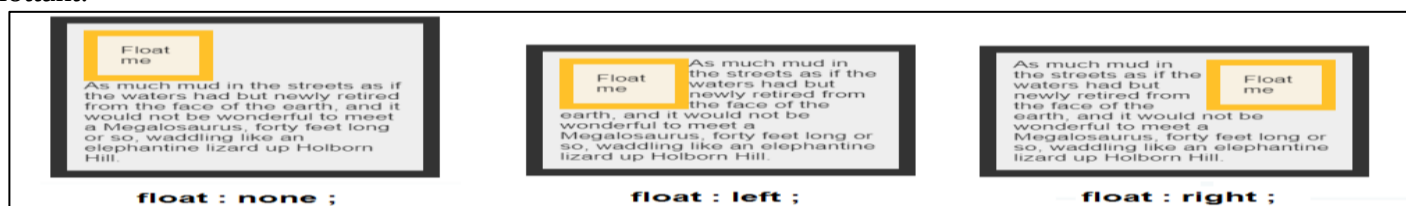
Avec le positionnement CSS, vous pouvez remplacer le flux normal du document.

La position : propriété spécifie le type de positionnement d'un élément.

. 1ère Partie : La propriété FLOAT

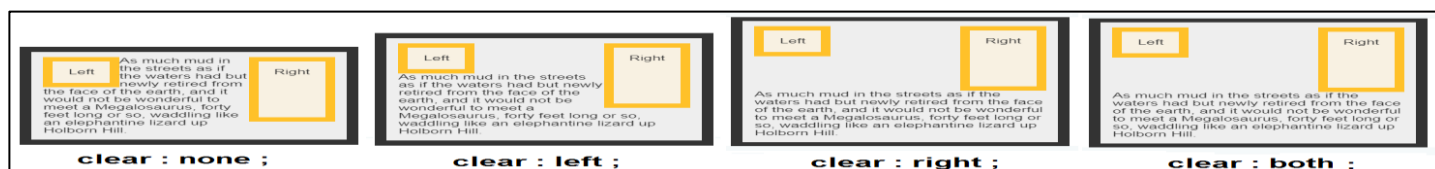
1. float :

La propriété **float** indique qu'un élément doit être retiré du flux normal et doit être placé sur le côté droit ou sur le côté gauche de son conteneur. Le texte et les autres éléments en ligne (*inline*) entoureront alors l'élément flottant.



2. clear :

La propriété **clear** indique si un élément peut être situé à côté d'éléments flottants qui le précèdent ou s'il doit être déplacé vers le bas pour être en dessous de ces éléments. La propriété **clear** s'applique aux éléments flottants comme aux éléments non-flottants.



➤ Travail à faire :

Observez les figures et appliquez les propriétés et les valeurs appropriées.

1. On considère le document html suivant :

```
<!doctype html>
<html>
<head>
  <title>Positionnement flottant</title>
  <link rel="stylesheet" href="style.css" >
  <meta charset="utf-8">
</head>
<body>
  <div class="conteneur">
    <div class="box">block2</div>
    <div class="box">block3</div>
    <div class="box">block4</div>
  </div>
</body>
</html>
```

➔ Complétez le code CSS suivant afin d'obtenir le résultat à droite.

<pre>div{ border: 1px solid rgba(0,0,0,.3); text-align : center; color : #fff; /*margin:5px; pour 4 Exps tableau*/ } .box{ float : left ; background: #3498db; margin : 5px; height: 40px; }</pre>	 blocs flottants
---	---

Remarque : Remarquez que les 3 blocs sortent de flux de leur parent, et par conséquent on se retrouve avec un conteneur de hauteur nulle.

Comment peut-on empêcher le dépassement des flottant afin d'obtenir le rendu suivant :	
Il suffit de définir la classe conteneur de cette manière :	<pre>.conteneur { overflow : hidden ; width : 300px ; }</pre>

- Nous allons produire le document html de la question précédente mais cette fois-ci nous allons identifier les éléments avec des classes différentes afin de les traiter séparément.

<pre>.block2{ float: left; background: #3498db; height: 40px; } .block3{ background: #d35400; height:60px; }</pre>	 float-left2
<pre>.block2{ float: left ; background: #3498db; height: 40px; } .block3{ float: right ; background: #d35400; height: 40px; } .block4{ clear: both ; background: #1abc9c; height: 40px; }</pre>	 float et clear
<pre>.block2{ float : right ; background: #3498db; height: 40px; } .block3{ clear: right; float: right; height: 40px; background: #d35400; } .block4{ clear: right; background: #1abc9c; height: 40px; }</pre>	 Float-et-clear-2

```

.block2{
    float: right;
    background: #3498db;
    height: 40px;
}
.block3{
    clear: right;
    float: right;
    background: #d35400;
    height: 40px;
}
.block4{
    clear : right ;
    float : right ;
    background: #1abc9c;
    height: 40px;
}

```



float-clear-3

. 2ème partie : La propriété position

Cette propriété peut avoir l'une des valeurs suivantes :

- **static** - Ceci est la valeur par défaut
- **relative**
- **fixed**
- **absolute**
- **sticky**

Les éléments sont ensuite positionnés à leur emplacement final avec les propriétés [top](#) [bottom](#) [left](#) [right](#)

- Position CSS : statique

Tous les éléments HTML sont positionnés de manière statique par défaut.

Les éléments positionnés statiquement ne sont pas affectés par les propriétés top, bottom, left et right.

Un élément avec **position: static;** est toujours positionné selon le flux normal de la page :

Cet élément <div> a la position : static ;

Exemple :

```

div.static {
    position: static;
    border: 3px solid #73AD21;
}

```

- Position CSS : relative

Un élément avec **position: relative;** est positionné par rapport à sa position normale dans le flux de documents.

La définition des propriétés haut, droite, bas et gauche ajustera l'élément par rapport à sa position normale. Le reste du contenu ne sera pas ajusté pour s'adapter à l'espace laissé par l'élément.

Ce <div> a une position : relative et est incliné de 30 px vers la droite de sa position normale

Exemple

```

div.relative {
    position: relative;
    left: 30px;
    border: 3px solid #73AD21;
}

```

- Position CSS : fixe

Un élément `position: fixed;` est positionné par rapport à la fenêtre d'affichage, ce qui signifie qu'il reste toujours au même endroit, même si la page défile. Les propriétés `top`, `right`, `bottom` et `left` permettent de définir l'emplacement final de l'élément.

Un élément fixe ne laisse pas d'espace dans la page où il aurait normalement été situé.

Exemple :

```
div.fixed {  
  position: fixed;  
  bottom: 0;  
  right: 0;  
  width: 300px;  
  border: 3px solid #73AD21;  
}
```

- Position CSS : absolue

Un élément avec `position: absolute;` est positionné par rapport à l'ancêtre positionné le plus proche (avec une position autre que statique).

Cependant, si un élément positionné de manière absolue n'a pas d'ancêtre positionné, il utilise le corps du document et se déplace avec le défilement de la page.

Remarque : les éléments positionnés de manière absolue sont supprimés du flux normal du document et peuvent chevaucher d'autres éléments.

```
div.relative {  
  position: relative;  
  width: 400px;  
  height: 200px;  
  border: 3px solid green;  
}
```

```
div.absolute {  
  position: absolute;  
  top: 80px;  
  right: 0;  
  width: 200px;  
  height: 100px;  
  border: 3px solid red;  
}
```

- Position CSS : sticky

La position de la boîte est calculée en fonction du flux normal du document. Ensuite, la boîte est décalée par rapport à son ancêtre de défilement le plus proche et par rapport à son bloc englobant selon les valeurs de `top`, `right`, `bottom` et `left`. Dans tous les cas, y compris avec les éléments `table`, cela n'affecte pas la position des autres éléments.

Cette valeur entraîne toujours la création d'un nouveau contexte d'empilement.

Création d'un site web interactif

Fiche n°9

TRANSITIONS

Les transitions s en CSS ?

Les transitions permettent de passer d'un affichage à un autre pour un même objet, d'un contenu à un autre, de façon progressive.

1. Syntaxe de la propriété :

▪ transition-property :

Indique à quelle propriété on fournit une transition. Par défaut **all**, c'est-à-dire toutes les propriétés.

Par exemple pour changer la couleur progressivement :

```
transition-property: color;
```

▪ transition-duration :

Définit la durée totale du temps de la transition. Par défaut la valeur est 0, il n'y a aucune transition.

Exemple, pour une durée d'une demi-seconde :

```
transition-duration: 0.5s;
```

▪ transition-timing-function :

Définit la forme de la progression dans le temps, avec des accélérations possibles durant la transition.

➔ Liste des valeurs possibles:

- **ease**: Par défaut, Cela donne un départ et une arrivée ralentis.
- **linear**: Une fonction linéaire,
- **ease-in**: Départ ralenti.
- **ease-out**: Arrivée ralentie.
- **ease-in-out**: Départ et arrivée ralentis.

▪ transition-delay

Indique à quel moment démarre la transition (par rapport à l'évènement qui la déclenche).

Par exemple pour un délai de réaction d'une seconde :

```
transition-delay: 1s;
```

Par défaut le délai est de 0.

2. Transition :

Les quatre propriétés peuvent être réduites à une seule.

On place dans l'ordre : **transition-property**, **transition-duration**, **transition-timing-function**, **transition-delay**.

Pas de virgules pour séparer les valeurs, comme usuel.

```
transition: all 2s ease 0;
```

On peut omettre les dernières valeurs.

3. Utiliser des listes :

On peut avoir une liste de groupes de propriétés séparées par une virgule :

```
transition: color 0.2s ease 1s, top 2s ease-in 0;
```

Les propriétés élémentaires comme la forme compacte peuvent comporter des listes séparées par une virgule :

```
transition-property: color, top;
```

```
transition-duration:1s, 5s;
```

Les éléments des listes sont associés par leur position dans la liste. Une durée d'une seconde est attribuée à la propriété *color*, une durée de 5 secondes à la propriété *top*.

4. Exemple :

Page.html :

```
<!DOCTYPE html>
<html lang="en">
<head>
<link rel="stylesheet" href="tr2.css">
  <meta charset="UTF-8">
  <title>Document</title>
</head>
<body>
  <div>3SI</div>
</body>
</html>
```

tr2.css :

```
div{
  color: white;
  background-color: red;
  width: 300px;
  height: 300px;
  text-align: center;
  position: relative;
  top: 0;
  left: 0;
  transition:left,border-radius 5s,10s ;
}
div:hover{
  left:1000px;
  border-radius: 50%;
}
```

Création d'un site web interactif

Fiche n°10

ANIMATION

Pour créer une animation CSS on doit :

- créer et nommer l'animation (avec **@keyframes**)
- attacher cette animation à un élément (avec **animation-name**)

1. Notre première animation :

On commence par créer une animation :

```
@keyframes taille {
  from {width: 0px;}
  to {width: 200px;}
}
```

On utilise la règle **@keyframes** et on nomme l'animation **taille**.

Remarque : Il existe quelques règles avec le signe **@** (nommées **at-rules**) qui permettent d'encapsuler plusieurs règles pour le processeur CSS du navigateur, par exemple **@media**, **@page**...

Ici, on change la largeur (**width**) de (**from**) 0px à (**to**) 200px.

On attache ensuite cette animation à un élément :

```
div {
  animation-name: taille;
  height: 100px;
  width: 200px;
  background-color: blue;
  animation-duration: 4s;
}
```

On utilise :

- **animation-name** : pour désigner l'animation utilisée. (*obligatoire*)
- **animation-duration** : pour définir la durée de l'animation (par défaut elle est de 0 seconde). (*obligatoire*)

2. Plusieurs étapes :

L'intérêt des animations est de pouvoir gérer des étapes intermédiaires. Voici une nouvelle version de l'animation :

```
@keyframes taille {
  0% {width: 0px;}
  50% {width: 300px;}
  100% {width: 200px;}
}
```

On a 3 étapes :

- le départ (0%) avec une largeur de 0px
- la moitié (50%) avec une largeur de 300px
- l'arrivée (100%) avec une largeur de 200px

3. Plusieurs propriétés :

On peut aussi changer plusieurs propriétés :

```
@keyframes taille {
  0% {width: 0px; background-color: red;}
  25% {width: 50px; background-color: yellow;}
  50% {width: 100px; background-color: green;}
  75% {width: 150px; background-color: pink;}
  /* 100% {width: 200px; background-color: blue;} devient facultatif car
  l'état 100% est déjà l'état du div "final".*/
}
```

4. Démarrage différé :

Dans les exemples ci-dessus, l'animation commence dès le chargement, ce qui n'est pas toujours souhaitable. On dispose de la propriété **animation-delay** pour différer le départ de la durée voulue.

En poursuivant notre exemple ça pourrait donner ce code :

```
div {
  animation-name: taille;
  height: 100px;
  width: 200px;
  background-color: blue;
  animation-duration: 4s;
  animation-delay: 2s;
}
```

5. Nombre d'exécutions :

Par défaut, une animation est exécutée une seule fois, si on en veut plus, il faut utiliser la propriété **animation-iteration-count**. Toujours avec notre exemple :

```
div {
  animation-name: taille;
  height: 100px;
  width: 200px;
  background-color: blue;
  animation-duration: 4s;
  animation-iteration-count: 2;
}
```

Remarque :

Si on veut un nombre d'exécutions infini, au lieu de mettre une valeur on utilisera **infinite**.

6. Sens de l'animation :

Par défaut une animation va dans le sens normal mais on peut aussi l'obliger à aller dans l'autre sens en utilisant la propriété **animation-direction**. On dispose de deux valeurs :

- **reverse** : on va dans le sens inverse.
- **alternate** : on alterne d'un sens à l'autre (dans ce cas évidemment il faut prévoir au moins deux exécutions).

```
div {
  height: 100px;
  width: 200px;
  background-color: blue;
  animation-name: taille;
  animation-duration: 4s;
  animation-iteration-count: infinite;
  animation-direction: alternate;
}
```

7. Progression de l'animation :

On peut modifier l'animation avec la propriété **animation-timing-function** et ces valeurs :

- **ease** : début rapide, puis ça accélère, puis ça ralentit à la fin (c'est l'effet par défaut).
- **linear** : même vitesse du début à la fin.
- **ease-in** : début lent.
- **ease-out** : fin lente.
- **ease-in-out** : début et fin lents.

8. Le style avant et après l'animation :

Le style avant et après l'animation est défini par défaut par les règles de l'élément. On a la possibilité de changer ce comportement en utilisant la propriété **animation-fill-mode**. On dispose de ces valeurs :

- **none** : comportement par défaut (en gros ça sert à rien)
- **forwards** : on garde les valeurs calculées lors de la dernière étape
- **backwards** : on garde les valeurs calculées lors de la première étape

L'interprétation peut être délicate selon les répétitions et sens des animations.

Par exemple avec ce code :

```
@keyframes taille {
  0% {width: 0px ;}
  100% {width: 200px ;}
}
div {
  height: 100px;
  width: 200px;
  background-color: blue;
  animation-name: taille;
  animation-duration: 4s;
  animation-delay: 2s;
  animation-fill-mode: backwards;
}
```

Au départ et pendant le délai de démarrage, la largeur sera de *0px* qui correspond à la première étape (à 0%) parce qu'on a utilisé la valeur **backwards** pour **animation-fill-mode**.

Création d'un site web interactif

Fiche n°11

JavaScript

→ C'est quoi le JavaScript ?

Le JavaScript est un langage de script qui offre de nombreuses possibilités. Sa présence croissante sur le web permet de naviguer sur des sites de plus en plus interactifs et rapides.

Il permet, entre autres, d'introduire sur une page web ou HTML des petites animations ou des effets. Le langage JavaScript se distingue des langages serveurs par le fait que l'exécution des tâches est opérée par le navigateur lui-même, sur l'ordinateur de l'utilisateur, et non sur le serveur web. Il s'active donc généralement sur le poste client plutôt que côté serveur.

→ Quels sont les principaux usages de JavaScript ?

Le langage JavaScript est principalement employé pour améliorer l'ergonomie d'un site Internet et/ou d'une interface applicative utilisateur. Il sert également à :

- ❖ Stocker des valeurs utiles dans des variables.
- ❖ Faire des opérations sur des chaînes de texte.
- ❖ Écouter les événements (comme un clic et mouvement de la souris)
- ❖ Modifier le HTML et le CSS de votre page après le chargement de la page
- ❖ Bouger les choses sur l'écran de manière interactive.
- ❖ Créer des jeux géniaux qui fonctionnent dans le navigateur
- ❖ Communiquer des données entre le serveur et le navigateur
- ❖ Permet d'interagir avec une webcam, un microphone et d'autres appareils

→ Ordre d'exécution du JavaScript !

Quand le navigateur rencontre un bloc de JavaScript, il l'exécute généralement dans l'ordre, de haut en bas.

→ Comment ajouter du JavaScript à votre page ?

Le JavaScript est appliqué à la page HTML un peu comme le CSS. Les éléments **<link>** permettent d'appliquer des feuilles de style externes au HTML, le CSS internes utilise l'élément **<style>**. Pour le JavaScript il y a une seule façon, avec l'élément **<script>**.

❖ JavaScript externe :

Généralement, un script est écrit dans un fichier « **.js** » à part. Pour exécuter un script depuis un fichier dans la page web, il suffira d'utiliser **<script>** avec un attribut « **src** » pointant vers le fichier du script en utilisant l'URL du fichier

```
<script src="chemin/vers/le/script.js"></script>
```

❖ JavaScript interne :

Il est également possible d'insérer du code JavaScript directement dans la balise `<script>` sans fournir d'attribut « src » .

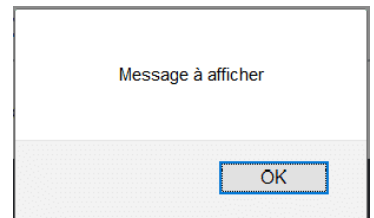
Exemple

```
<script>
window.addEventListener('load', function () {
    console.log('Cette fonction est exécutée une fois
quand la page est chargée. ');
});
</script>
```

Pour éviter des problèmes de chargement sur les pages, il est conseillé de placer les éléments `<script>` juste avant la fermeture de l'élément `<body>`.

→ La boîte de dialogue `alert()`

`alert()` est une instruction simple, appelée fonction, qui permet d'afficher une boîte de dialogue contenant un message. Ce message est placé entre apostrophes, elles-mêmes placées entre les parenthèses de la fonction.



→ La syntaxe Javascript

La syntaxe du Javascript n'est pas compliquée. De manière générale, les instructions doivent être séparées par un point-virgule que l'on place à la fin de chaque instruction :

```
<script>
    instruction_1;
    instruction_2;
    myFunction();
</script>
```

```
<script>
alert('Bonjour');
// la fonction affiche une boîte de
dialogue avec "Bonjour"
</script>
```

→ Les commentaires

```
<script>
    instruction_1; // Ceci est ma première instruction
    instruction_2;
    /* La troisième instruction ci-dessous,
       avec un commentaire sur deux lignes */
    instruction_3;
</script>
```

→ Les variables en JavaScript

Une variable consiste en un espace de stockage, qui permet de garder en mémoire tout type de données. La variable est ensuite utilisée dans les scripts. Une variable contient seulement des caractères alphanumériques, le \$ (dollar) et le _ (underscore) ; elle ne peut pas commencer

par un chiffre ni prendre le nom d'une fonction existante de Javascript. On crée la variable et on lui affecte (ou attribue) une valeur :

```
<script>
    var myVariable;
    myVariable = 2;
</script>
```

ou

```
<script>
    var myVariable = 2;
</script>
```

→ Les types de variables

Une variable peut être de type numérique, une chaîne de caractères ou booléen.

```
<script>
var text = 'J\'écris mon texte ici';
/* Avec des apostrophes, le \ sert à échapper une apostrophe
intégrée dans le texte, pour ne pas que Javascript pense
que le texte s'arrête là.*/
</script>
```

→ Les opérateurs arithmétiques types de variables

On peut utiliser 5 opérateurs arithmétiques : l'addition (+), la soustraction (-), la multiplication (*), la division (/) et le modulo (%).

→ La concaténation

Une concaténation consiste à ajouter une chaîne de caractères à la fin d'une autre, comme dans cet exemple :

```
<script>
var hi = 'Bonjour ', name = 'toi',
result;
result = hi + name;
alert(result);
// Affiche : « Bonjour toi »
</script>
```

```
<script>
    var text = 'Bonjour ';
    text += 'toi';
    alert(text);
    // Affiche « Bonjour toi ».
</script>
```

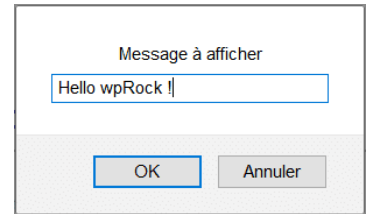
On peut convertir un nombre en chaîne de caractères avec l'astuce suivante :

```
<script>
    var text, number1 = 4, number2 = 2;
    text = number1 + ' ' + number2;
    /* on ajoute une chaîne de caractères vide entre les
    deux nombres pour permettre une concaténation sans
    calcul */
    alert(text); // Affiche : « 42 »
</script>
```

→ La fonction `prompt()`, avec concaténation et calcul

La boîte de dialogue `prompt()` permet d'afficher un message, un champ à remplir (input de type texte) et un bouton « OK ». Cette fonction retourne la valeur qui a été entrée dans le champ par l'utilisateur.

```
<script>
  var userName = prompt('Entrez votre prénom :');
  alert(userName);
  // Affiche le prénom entré par l'utilisateur
</script>
```



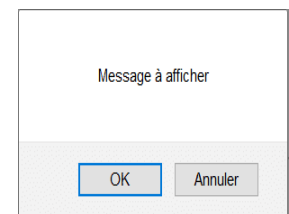
On peut aussi se servir de la fonction `prompt()` pour un calcul

```
<script>
  var first, second, result;
  first = prompt('Entrez le premier chiffre :');
  second = prompt('Entrez le second chiffre :');
  result = parseInt(first) + parseInt(second);
  /* la fonction parseInt() transforme la chaîne de
  caractères en nombre */
  alert(result);
</script>
```

→ La boîte de dialogue `confirm()`

La boîte de dialogue `confirm()` permet d'afficher un message et deux boutons : un bouton « OK » et un bouton « Annuler ». Cette fonction retourne une valeur booléenne qui vaut « true » si c'est le bouton OK qui est cliqué et retourne « false » si c'est le bouton « Annuler » qui est cliqué.

```
if ( confirm( "Message à afficher" ) ) {
  // Code à exécuter si le l'utilisateur clique sur "OK"
} else {
  // Code à exécuter si l'utilisateur clique sur "Annuler"
}
```



Création d'un site web interactif

Fiche n°12

JavaScript Notion de base

→ Quels sont les opérateurs de comparaison a utilisés ?

==	égal à	>	supérieur à
!=	différent de	>=	supérieur ou égal à
===	contenu et type de variable égal à	<	inférieur à
!==	contenu ou type de variable différent de	<=	inférieur ou égal à

```
<script>
var number1 = 2, number2 = 2, number3 = 4, result ;
result = number1 == number2;    /* la variable result
reçoit le résultat de comparaison de contenu de deux
variables */
alert(result);    /* la condition est donc vérifiée car
les deux variables contiennent bien la même valeur*/
result = number1 == number3;
alert(result);    // la condition n'est pas vérifiée car
2 est différent de 4
result = number1 < number3;
alert(result);    // la condition est vérifiée car 2 est
bien inférieur à 4
</script>
```

→ Quels sont les opérateurs logiques a utilisés ?

- ❖ **&&** : qui signifie ET avec par exemple : **var1 && var2**. Cet opérateur vérifie la condition lorsque toutes les variables (type booléen) qui lui sont passées valent vrai (true).
- ❖ **||** : qui signifie OU avec par exemple : **var1 || var2**. Cet opérateur est plus souple car il renvoie vrai (true) si une variable qui lui est soumise contient vrai (true), qu'importe les autres variable.
- ❖ **!** : qui signifie NON avec par exemple : **!var**. Cet opérateur se différencie des deux autres car il ne prend qu'une seule valeur à la fois. S'il se nomme « NON » c'est parce que sa fonction est d'inverser la valeur qui lui est passée, ainsi true deviendra false et inversement.

Nb : **var** de type booléen ou un résultat d'une fonction qui renvoie vrai ou faux ou toute une instruction de comparaison

```
<script>
var number1 = 2, number2 = 2, number3 = 4, result;
result = (number1 < number2) && (number1==number2);
alert(result);
result = (number1 == number3) || !(number1==number2 );
alert(result);
</script>
```

→ La structure « if.. else »

La condition **if** est une structure conditionnelle limitée par définition puisqu'elle ne nous permet d'exécuter un bloc de code que dans le cas où le résultat d'un test est évalué à **true** mais elle ne nous offre aucun support dans le cas contraire.

```
<script>
var a = 1,b=2;
if (a > b) {
    result = 'a est le plus grand';
}
alert(result);
/* La fonction alert n'affiche rien car
la condition non vérifiée*/
</script>
```

```
<script>
if (confirm('Voulez-vous exécuter le
code Javascript de cette page ?')) {
    alert('Le code a bien été exécuté !');
}
</script>
```

La structure conditionnelle **if...else** (« si... sinon » en français) va être plus complète que la condition **if** puisqu'elle va nous permettre d'exécuter un premier bloc de code si un test renvoie **true** ou un autre bloc de code dans le cas contraire.

```
<script>
var a = 1,b=2;
if (a > b) {
    result = 'a est le plus grand';
}
else
{
    result = 'b est le plus grand';
}
alert(result);
// la fonction alert affiche "condition
pas vérifiée"
</script>
```

```
<script>
if (confirm('Pour accéder à ce site vous
devez être une fille, cliquez sur
"OK" si c\'est le cas.')) {
    alert('Vous allez être redirigé vers le
site.');
```

La condition **if...else if...else** (« si...sinon si...sinon ») est une structure conditionnelle encore plus complète que la condition **if...else** puisqu'elle va nous permettre cette fois-ci de générer et de prendre en charge autant de cas que l'on souhaite

```
<script>
var a = 1,b=2;
if (a > b) {
    result = 'a est le plus grand';
}
else if (a<b)
{
    result = 'b est le plus grand';
}
else
{
    result = 'a et b sont égaux';
}
alert(result);
// la fonction alert affiche "b le
plus grand"
</script>
```

```
<script>
var etage = parseInt(prompt("Entrez l'étage
où l'ascenseur doit se rendre (de -2 à 30)
:"));
if (etage == 0) {
    alert('Vous vous trouvez déjà au rez-de-
chaussée.');
```

→ La structure « switch »

La structure **switch** commence par évaluer l'expression fournie (cette évaluation ne se produit qu'une fois). Si une correspondance est trouvée, le programme exécutera les instructions associées. Si plusieurs cas de figure correspondent, le premier sera sélectionné (même si les cas sont différents les uns des autres).

```
switch (expression) {
  case valeur1:
    // Instructions à exécuter lorsque le résultat
    // de l'expression correspond à valeur1
    instructions1;
    [break;]
  case valeur2:
    // Instructions à exécuter lorsque le résultat
    // de l'expression correspond à valeur2
    instructions 2;
    [break;]
  ...
  case valeurN:
    // Instructions à exécuter lorsque le résultat
    // de l'expression à valeurN
    instructionsN;
    [break;]
  [default:]
    // Instructions à exécuter lorsqu'aucune des
    // valeurs
}
}
```

```
switch (fruit=prompt('donner le nom de
fruit que vous voulez'))
{
  case "Oranges":
    alert ("Oranges : 2 Dt le kilo.");
    break;
  case "Pommes":
    alert("Pommes : 3,5 DT le kilo.");
    break;
  case "Bananes":
  case "Cerises":
    alert ("Bananes et Cerises : 9,5 Dt le
kilo.");
    break;
  default:
    alert("Désolé, nous n'avons plus de " +
fruit + ".");
}
```

Création d'un site web interactif

Fiche n°13

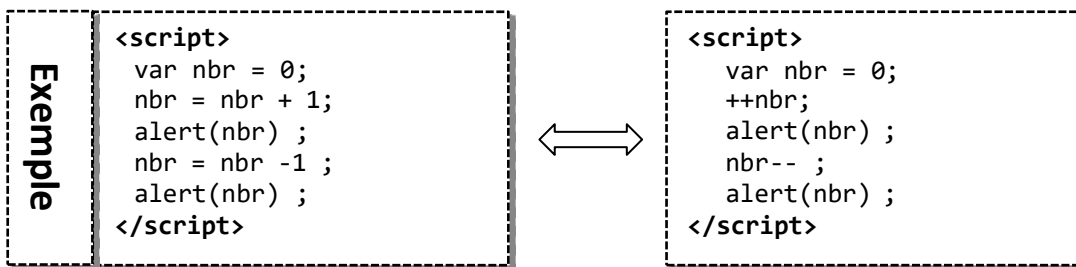
JavaScript Les structures répétitives

Une boucle est une structure analogue aux structures conditionnelles sauf qu'ici il s'agit de répéter une série d'instructions. La répétition se fait jusqu'à ce qu'on dise à la boucle de s'arrêter. À chaque fois que la boucle se répète on parle d'itération (répétition).

On utilise un itérateur qui modifie la valeur de départ de la boucle à chaque nouveau passage jusqu'au moment où la condition de sortie est vérifiée. Bien souvent, on incrémentera ou décrémentera la valeur de départ à l'aide des opérateurs d'incrémentement et décrémentement.

→ Les opérateurs d'incrémentement et décrémentement

Les opérations d'incrémentement et de décrémentement vont principalement être utilisées avec les boucles en JavaScript. L'incrémentement permet d'ajouter une unité à un nombre au moyen d'une syntaxe courte. À l'inverse, la décrémentement permet de soustraire une unité.



→ La boucle POUR « for »

Une boucle **for** contient trois « phases » à l'intérieur du couple de parenthèses : une phase d'initialisation, une phase de test (condition de sortie) et une phase d'itération (généralement une incrémentement). Chaque phase est séparée des autres par un point-virgule.

Syntaxe	<pre><script> for(initialisation; condition; incrémentement) { instruction_1; instruction_2; instruction_3; } </script></pre>	Exemple	<pre><script> var i=0; for (i=0;i<=6;i++) { console.log ("le nbr est:"+ i); }} </script></pre>
Exemple	<pre><script> for (var prenoms = '', prenom; true;) { /*ici sans incrémentation nécessaire mais avec un point-virgule obligatoire après la condition true */ prenom = prompt('Entrez un prénom :'); if (prenom) { prenoms += prenom + ' '; } else { break; } } alert(prenoms); </script></pre>		

On peut se servir de l'incrémentation pour compter le nombre de prénoms :

Exemple

```
<script>
for (var i = 0, prenoms = '', prenom; true; i++) {
  prenom = prompt('Entrez un prénom :');
  if (prenom) { prenoms += prenom + ' '; }
  else { break; } }
alert('Il y a ' + i + ' prénoms :\n\n' + prenoms);
// Les \n servent à faire des sauts de ligne
</script>
```

→ La boucle TANT QUE «while»

L'instruction **while** permet de créer une boucle qui s'exécute tant qu'une condition de test est vérifiée. La condition est évaluée avant d'exécuter l'instruction contenue dans la boucle.

Syntaxe

```
<script>
while (condition) {
  instruction_1;
  instruction_2;
  instruction_3;
}
</script>
```

Exemple

```
<script>
var nbr = 1;
while (nbr < 10) {
  nbr++;
}
alert(nbr); // Affiche : « 10 »
</script>
```

Exemple

```
<script>
var pseudos = '', pseudo,
continuer=confirm('vous devez saisir un pseudo ');
while (continuer) {
  pseudo = prompt('Entrez un pseudo :');
  if (pseudo) {
    pseudos += pseudo + ' ';
    /* concatination de nouveau pseudo et un espace avec l'ancienne
    valeur pseudos */
  } else {
    continuer = false;
    // Aucun pseudo n'a été saisi, donc la boucle se termine
    // On peut utiliser break pour quitter la boucle
  }
}
alert(pseudos); // Affiche tous les pseudos
</script>
```

→ La boucle répéter..jusqu'à «do.. while»

L'instruction **do...while** crée une boucle qui exécute une instruction jusqu'à ce qu'une condition de test ne soit plus vérifiée. La condition est testée après que l'instruction soit exécutée, le bloc d'instructions défini dans la boucle est donc exécuté au moins une fois.

Syntaxe

```
<script>
do {
  instruction_1;
  instruction_2;
  instruction_3;
}
While (condition)
</script>
```

Exemple

```
<script>
var i = 0;
do {
  i += 1;
  console.log(i);
} while (i < 5); // 5 sera afficher
</script>
```

Exemple

```
<script>
  let somme = 0;
  let nbr = 0;
  /* L'instruction let permet de déclarer une variable dont la portée est celle
  du bloc courant, éventuellement en initialisant sa valeur */
  do {
    somme += nbr;
    nbr = parseInt(prompt('Entrer un nombre: '));
  } while(nbr >= 0);
  alert('la somme est ' + somme);
</script>
```

TP

Utilisons la boucle **do-while** pour développer un jeu de devinettes simple. Le script génère un entier aléatoire entre 1 et 12. Vous devez deviner le nombre en faisant des suppositions jusqu'à ce que le nombre que vous choisissez corresponde au nombre choisi par le script.

```
<script>
// Génération de nombre secret entre deux valeurs MIN et MAX
const MIN = 1;
const MAX = 12;
let nombresecret = Math.floor(Math.random() * (MAX - MIN + 1)) + MIN;

/* Déclaration et initialisation d'un nombre secret
La fonction Math.floor(x) renvoie le plus grand entier qui est inférieur ou égal à un
nombre x
La fonction Math.random() renvoie un nombre flottant pseudo-aléatoire compris dans l'intervalle [0, 1[ (ce qui
signifie que 0 est compris dans l'intervalle mais que 1 en est exclu) selon une distribution approximativement
uniforme sur cet intervalle. Ce nombre peut ensuite être multiplié afin de couvrir un autre intervalle */

let supposition = 0; // Enregistrement de nombre de suppositions
let indice = ' '; // Enregistrement d'indice
let nbr = 0;
do {
  // Valeur introduite par l'utilisateur
  let val = prompt('Svp entrer un nombre entre ' + MIN + ' et ' + MAX + indice);
  nbr=parseInt(val);
  // Incrémentation de nombre de supposition
  supposition++;
  // Vérification avec le nombre secret
  if (nbr > nombresecret) {
    indice = ', plus petit que ' + nbr;
  } else if (nbr < nombresecret) {
    indice = ', plus grand que ' + nbr;
  } else if(nbr == nombresecret) {
    alert('Bravo! vous avez réussi après ' + supposition + ' suppositions. ');
  }
} while (nbr != nombresecret);
</script>
```

Création d'un site web interactif

Fiche n°14

JavaScript Fonctions et événements

→ C'est qui une fonction?

- ✓ Une fonction est un ensemble d'instructions effectuant une tâche ou calculant une valeur. Afin d'utiliser une fonction, il est nécessaire de l'avoir auparavant définie au sein de la portée dans laquelle on souhaite l'appeler.
- ✓ Il y a les fonctions ou variables natives (déjà existantes), mais on peut aussi en créer de nouvelles, selon la structure suivante

Syntaxe	<pre><Script> // Le terme "function" est obligatoire pour déclarer une fonction function nom_fonction(arguments) { // Le code que la fonction va devoir exécuter } </Script></pre>
Exemple	<pre><script> function Multiplier_par_deux() { var result = parseInt(prompt('Donnez le nombre à multiplier par 2 :')); alert(result * 2); } Multiplier_par_deux(); // On appelle la fonction créée alert('Vous avez déjà appelé la fonction une fois !'); Multiplier_par_deux(); // Et appelle de nouveau la fonction </script></pre>

→ Les variables locales et globales

Toute variable déclarée dans une fonction n'est utilisable que dans cette même fonction. Ces variables spécifiques à une seule fonction ont un nom : les variables locales. Déclarées en dehors des fonctions, on parle de variables globales.

Exemple	<pre><script> <script> var message = 'Ici la variable globale !'; function showMsg() { var message = 'Ici la variable locale !'; alert(message); } showMsg(); // On affiche la variable locale alert(message); // Puis la variable globale </script></pre>
----------------	--

→ Les arguments

Les paramètres donnés lors de l'appel d'une fonction sont appelés les *arguments* de la fonction. Les arguments sont passés *par valeur* (*by value* en anglais). Si la fonction modifie la valeur d'un argument, ce changement ne se répercute pas en dehors de la fonction.

Exemple

```
<script>
var message = 'Ici la variable globale !';
function showMsg() {
var message = 'Ici la variable locale !';
alert(message);
}
showMsg(); // On affiche la variable locale
alert(message); // Puis la variable globale
</script>
```

TP**TP1**

```
<!DOCTYPE>
<html lang="fr-FR">
<head>
  <title>getElementById example</title>
</head>
<body>
<script>
  function changeColor(newColor) {
    var element = document.getElementById('para');
    element.style.color = newColor; }
</script>
  <p id="para">Paragraphe à colorier</p>
  <button onclick="changeColor('blue');">bleu</button>
  <button onclick="changeColor('red');">rouge</button>
</body>
</html>
```

TP2

```
<!DOCTYPE>
<html lang="fr-FR"><head></head>
<body>
  <div style="text-align:center">Cliquer sur les boutons dans le bon
  ordre.<br>
  <input type="button" id="btn1" onClick="ajouter(this)" value="est">
  <input type="button" id="btn2" onClick="ajouter(this)" value="soleil">
  <input type="button" id="btn3" onClick="ajouter(this)" value="chaud">
  <input type="button" id="btn4" onClick="ajouter(this)" value="le"><br>
  <input type="text" id="text" size="60" readonly="true" style="background-
  color:#FFFFFF; border:#FFFFFF none 0px; text-align:center; color:#006600;
  font-weight:bold">
  <br><br>
  <input type="button" value="Recommencer" onClick="raz()"></div>
<script>
  function ajouter(objet) {
    document.getElementById("text").value+=objet.value;
    document.getElementById("text").value+=' ';
    objet.disabled=true;
    if (document.getElementById('text').value == "le soleil est chaud ") {
    document.getElementById('text').value="Le soleil est chaud."
    alert('BRAVO !!!');} }
  function raz() {
    document.getElementById('text').value="";
    document.getElementById('btn1').disabled=false;
    document.getElementById('btn2').disabled=false;
    document.getElementById('btn3').disabled=false;
    document.getElementById('btn4').disabled=false;
  }
</script> </body> </html>
```

Création d'un site web interactif

Fiche n°15

JavaScript La gestion du temps

→ C'est quoi La gestion du temps?

- ✓ Il s'agit en fait, en Javascript, du nombre de millisecondes écoulées depuis le 1er janvier 1970 à minuit. Cette manière d'enregistrer la date est très fortement inspirée du système d'horodatage utilisé par les systèmes Unix. La seule différence entre le système Unix et le système du Javascript, c'est que ce dernier stocke le nombre de millisecondes, tandis que le premier stocke le nombre de secondes. Dans les deux cas, ce nombre s'appelle un **timestamp**.
- ✓ Ce nombre ne nous sert vraiment qu'à peu de choses à nous, développeurs, car nous allons utiliser l'objet **Date** qui va s'occuper de faire tous les calculs nécessaires pour obtenir la date ou l'heure à partir de n'importe quel **timestamp**.

→ L'objet Date ?

L'objet **Date** nous fournit un grand nombre de méthodes pour lire ou écrire une date.

- ✓ Le constructeur : Commençons par le constructeur ! Ce dernier prend en paramètre de nombreux arguments et s'utilise de différentes manières. Voici les quatre manières de l'utiliser :

<code>new Date();</code>	Instancie un objet Date dont la date est fixée à celle de l'instant même de l'instanciation.
<code>new Date(timestamp);</code>	Permet de spécifier le timestamp à utiliser pour notre instanciation.
<code>new Date(dateString);</code>	Prend une chaîne de caractères qui doit être interprétable par la méthode parse() .
<code>new Date(année, mois, jour [, heure, minutes, secondes, millisecondes]);</code>	Permet de spécifier manuellement chaque composant qui constitue une date.

- ✓ Les méthodes de l'objet Date : Étant donné que l'objet Date ne possède aucune propriété standard, nous allons directement nous intéresser à ses méthodes qui sont très nombreuses

<code>getFullYear();</code>	Renvoie l'année sur 4 chiffres ;
<code>getMonth();</code>	Renvoie le mois (0 à 11) ;
<code>getDate();</code>	Renvoie le jour du mois (1 à 31) ;
<code>getDay();</code>	Renvoie le jour de la semaine (0 à 6, la semaine commence le dimanche) ;
<code>getHours();</code>	Renvoie l'heure (0 à 23) ;
<code>getMinutes();</code>	Renvoie les minutes (0 à 59) ;
<code>getSeconds();</code>	Renvoie les secondes (0 à 59) ;
<code>getMilliseconds();</code>	Renvoie les millisecondes (0 à 999).
<code>getTime();</code>	Renvoie le timestamp de la date de votre objet ;
<code>setTime();</code>	Permet de modifier la date de votre objet en lui passant en unique paramètre un timestamp.

Exemple 1

```
<script>
//Afficher l'heure actuelle du système
var d = new Date();
var heure = d.getHours() + ":" + d.getMinutes() + ":" + d.getSeconds();
alert(heure);
</script>
```

Exemple 2

```
<script>
// Exemple d'affichage de gestion d'un timestamp
var myDate = new Date('Sat, 04 May 1991 20:00:00 GMT+02:00');
//Creation d'un objet date avec timestamp comme argument
alert(myDate.getMonth()); // Affiche : 4
// Faites attention les mois commencent par 0
alert(myDate.getDate()); // Affiche le mois : 4
alert(myDate.getDay()); // Affiche le jour : 6
</script>
```

→ Tp : Création widget horloge avec javascript

TP (Horloge numérique)

```
<html>
<head>
<script>
function rafrachir(){
var t = 1000; // rafraîchissement en millisecondes
setTimeout('Affichehorloge()',t)
//setTimeout: déclenche des actions après un intervalle de temps donné
//On doit déclencher Affichehorloge() de nouveau après 1000 ms
//1 Millisecondes = 0.001 Secondes
}

function Affichehorloge() {
var date = new Date()
//Création de l'objet date
var h = date.getHours();
//Extraire l'heure de l'objet date
var m = date.getMinutes();
//Extraire les minutes de l'objet date
var s = date.getSeconds();
//Extraire les secondes de l'objet date
if( h < 10 ){ h = '0' + h; }
if( m < 10 ){ m = '0' + m; }
if( s < 10 ){ s = '0' + s; }
var time = h + ':' + m + ':' + s
document.getElementById('horloge').innerHTML = time;
//mise à jour du span horloge avec innerHTML
rafrachir();
//appel de la fonction pour le rafraîchissement des variables de nouveau
}
</script>
</head>
<body onload=Affichehorloge();>
<span id='horloge' style="background-color:black ;color:silver;font-size:100px;"></span>
</body>
</html>
```

innerHTML est une propriété de tout élément HTML qui désigne le contenu qui se trouve entre la balise entrante et la balise fermante. On l'utilise pour lire pour insérer dynamiquement un contenu.

Soit l'élément suivant:

```
<p id="parag">
un texte. </p>
```



On peut obtenir le contenu du paragraphe avec ce code:

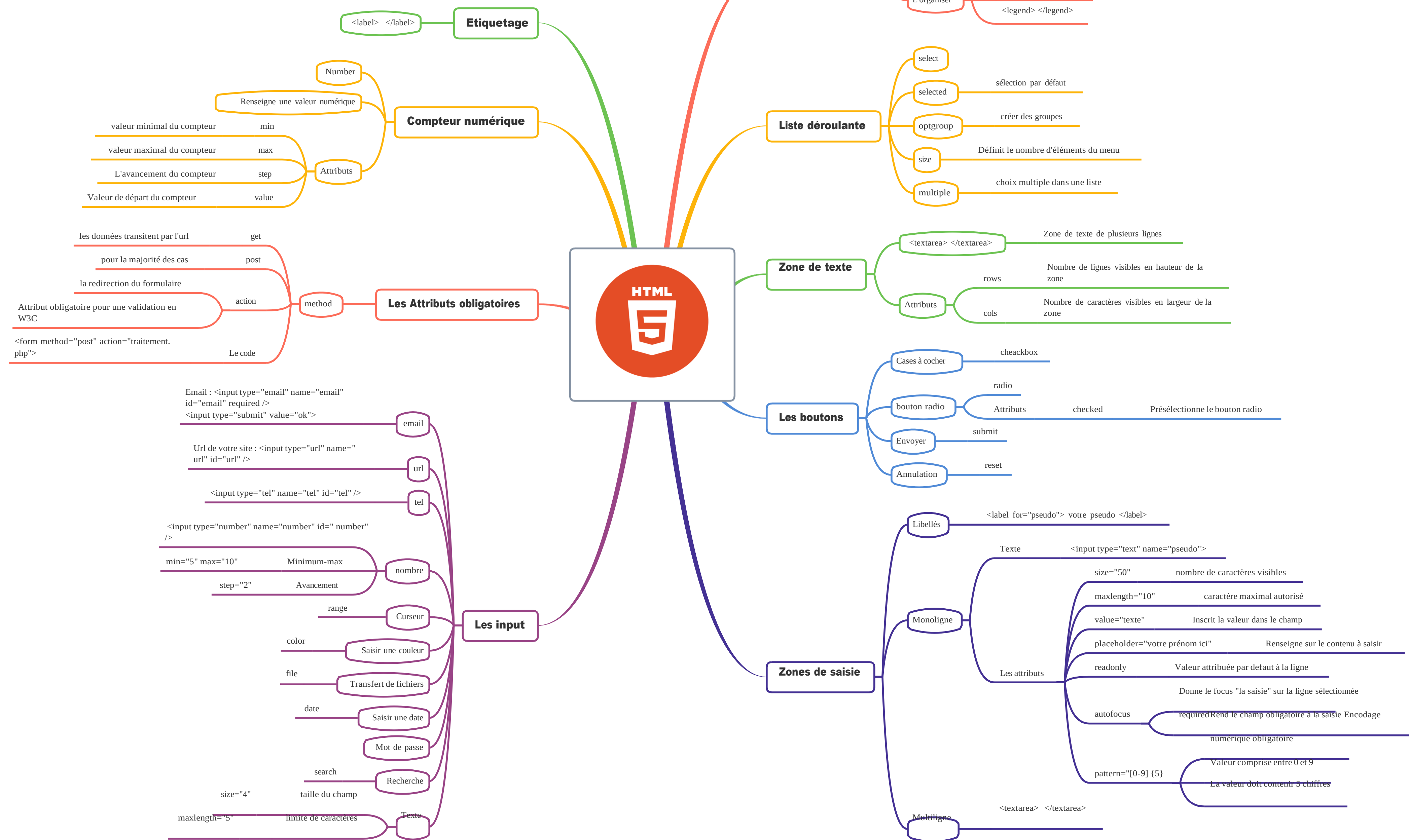
```
var x = document.getElementById("parag");
alert(x.innerHTML);
```

On peut remplacer dynamiquement le contenu du paragraphe :

```
var x = document.getElementById("parag");
x.innerHTML = "un autre texte";
```

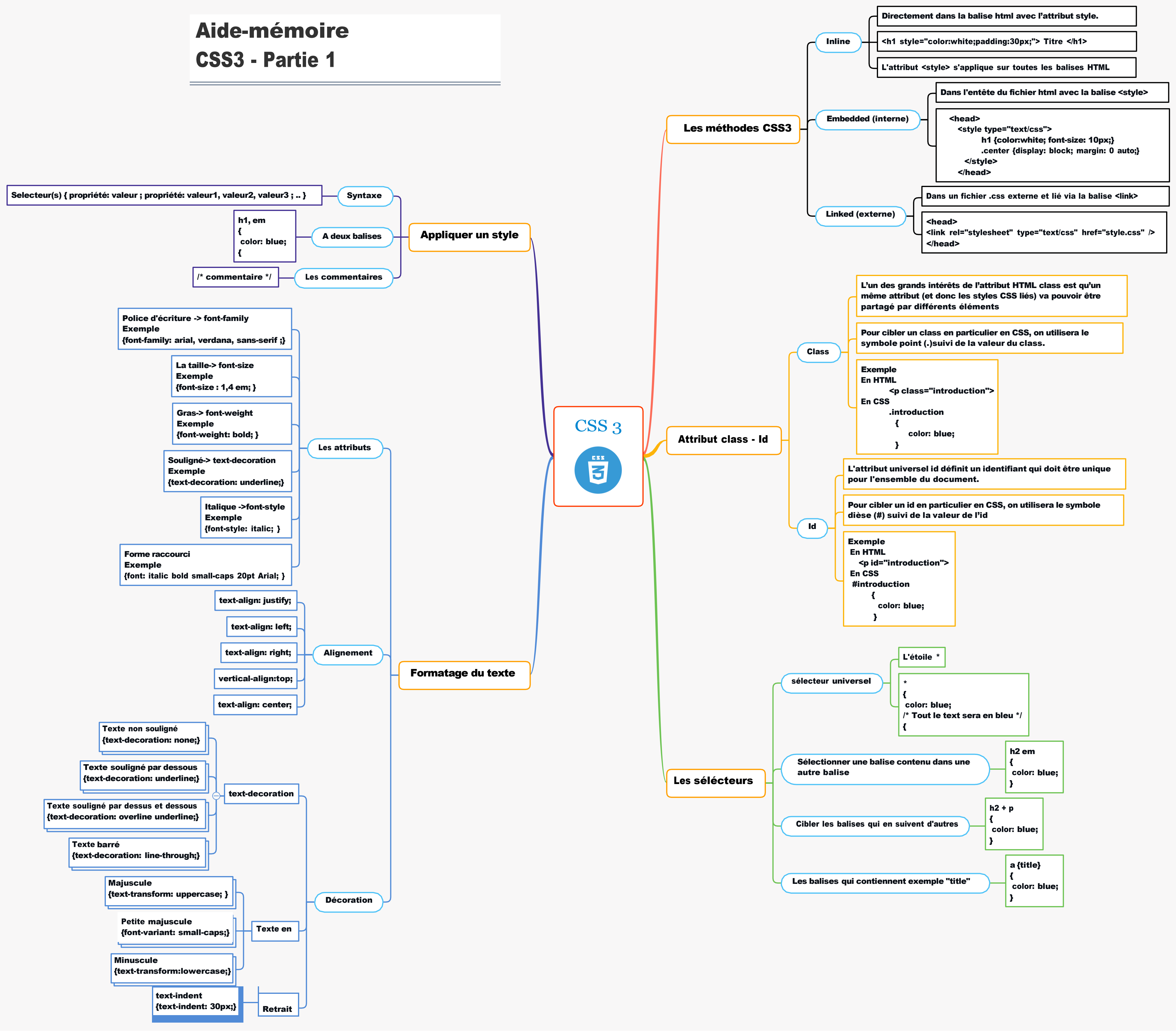
Aide-mémoire

Les formulaires - HTML5



Aide-mémoire

CSS3 - Partie 1



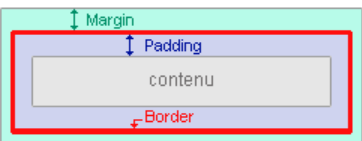
Les couleurs et les arrières-plans

Les Bordures

La propriété CSS border

Tous les éléments HTML acceptent une bordure Css, que cela soit des images, des div, des span ou des éléments de formulaires

Les bordures Css se situent entre les marges intérieures appelées padding et les marges extérieures appelées margin



On peut définir globalement les quatres bordures Css avec la propriété Css border si les bordures Css sont identiques.

Exemple
h1{ border : #FF00FF solid 2px; /* Définir toutes les bordures ensemble */ }

Les différentes bordures d'un élément

1 border-top :la bordure Css haute

border-top : #FF00FF solid 2px; /* Définition de la bordure Css haute */

2 border-bottom :la bordure Css basse

border-bottom : #FF00FF solid 2px; /* Définition de la bordure Css basse */

3 border-left :la bordure Css gauche

border-left : #FF00FF solid 2px; /* Définition de la bordure Css gauche */

4 border-right :la bordure Css droite

border-right : #FF00FF solid 2px; /* Définition de la bordure Css droite */

la couleur de la bordure

La propriété Css border-color permet de modifier la/les couleurs sur l'ensemble des bordures Css

Exemple
.nav{ border-color : crimson rgb(255,0,0) hsl(16,100%,50%) #FF00FF ; }

Les propriétés Css border-[left|right|top|bottom]-color permettent de modifier la couleur d'une bordure Css précise

Exemple
.nav { border-top-color : crimson; /* Couleur de la bordure Css haute */ }

l'épaisseur de la bordure

La propriété Css border-width permet de modifier la/les tailles sur l'ensemble des bordures Css

Exemple
.nav { border-width : 1px 2px 3px 4px; }

Les propriétés Css border-[left|right|top|bottom]-width permettent de modifier la taille d'une bordure Css précise

Exemple
.nav { border-left-width : 4px; /* Taille de la bordure Css gauche */ }

le style de la bordure

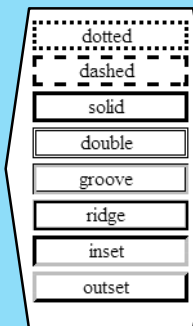
La propriété Css border-style permet de modifier le/les styles sur l'ensemble des bordures Css.

Exemple
.nav{border-style : dotted ; }

Les propriétés Css border-[left|right|top|bottom]-style permettent de modifier le style d'une bordure Css précise.

Exemple
.nav{ border-top-style : dotted; /* Style de la bordure Css haute */ }

Formes des styles



La Couleur d'avant-plan

La propriété CSS color permet de modifier la couleur d'avant-plan .

On peut définir les couleurs avec des valeurs hexadécimales comme (#ff0000), ou avec les noms des couleurs ("red"), ou avec des valeurs RGB (rgb(255,0,0)).

Exemple
h1 { color: #ff0000; }

La Couleur d'arrière-plan

La propriété background-color décrit la couleur d'arrière-plan des éléments.

Pour changer la couleur d'arrière-plan d'une page entière, il faudrait donc appliquer la propriété 'background-color' à l'élément <body>.On peut aussi appliquer une couleur d'arrière-plan à d'autres éléments y compris les titres et le texte.

Exemple
body { background-color: #FFCC66; }
h1 { color: #990000; background-color: #FC9804; }

L'image d'arrière-plan

La propriété CSS background-image sert à insérer une image d'arrière-plan.

Exemple
body { background-image: url("image.gif"); }

background-repeat

background-repeat: repeat-x; L'image se répète horizontalement

background-repeat: repeat-y; L'image se répète verticalement

background-repeat: repeat; L'image se répète horizontalement et verticalement

background-repeat: no-repeat; L'image ne se répète pas

background-attachment

La propriété background-attachment définit si l'image d'arrière-plan est fixe ou bien défile avec l'élément conteneur.

background-attachment: scroll; L'image défile avec la page (non bloquée)

background-attachment: fixed; L'image est bloquée

background-position

Par défaut, l'image d'arrière-plan se positionnera dans le coin supérieur gauche de l'écran. La propriété background-position permet de changer cette position.

Les coordonnées peuvent s'exprimer en pourcentages, ou en unités fixes (pixels, centimètres, etc.), ou on peut utiliser les mots-clés "top", "bottom", "center", "left" ou "right"

Exemple
body { background-color: #FFCC66; background-image: url("image.gif"); background-repeat: no-repeat; background-attachment: fixed; background-position: right bottom; }

La transparence

La propriété CSS opacity rend transparent les éléments d'une même zone

Exemple
h1 { color: white; opacity: 0.3; }

Le fond dégradé

Linéaire
background: linear-gradient

Circulaire
background: radial-gradient

Exemple
body { background: radial-gradient(red, blue);}

Combiner des valeurs

On peut obtenir le même résultat avec background en une seule ligne de code

[background-color] | [background-image] | [background-repeat] | [backgroundattachment] | [background-position]

background: #FFCC66 url("butterfly.gif") no-repeat fixed right bottom;